EnCorelib user guide

End covers for the initial value problem

Bingwei Zhang Chee Yap

November 2026

Contents

1 What EnCorelib computes **2**

2 Installing **2**

2.1 What you need . . . 2
2.2 Building CAPD . . . 3
2.3 Building EnCorelib . . . 3
2.4 Checking the build . . . 4

3 A first run **4**

3.1 An example from the paper . . . 4
3.2 The same run without make . . . 5
3.3 Output levels and files . . . 5

4 A problem of your own **7**

4.1 A `.mk` file . . . 7
4.2 Writing the right-hand side . . . 9
4.3 Constants . . . 9
4.4 The form of the expression . . . 9
4.5 Mistakes in the input . . . 9

5 Following a long run **10**

5.1 One line per sub-box . . . 10
5.2 Inside one sub-box . . . 11
5.3 A long run . . . 12

6 Choosing the options **13**

6.1 The error bound . . . 13
6.2 The horizon . . . 13
6.3 The enclosure step: `stepB` . . . 13
6.4 The refinement operators: `method` . . . 14
6.5 The Taylor order . . . 15
6.6 Boundary mode . . . 15

6.7 The remaining options 15
6.8 In short 16

7 When a run fails **16**

8 Reproducing the paper **17**

9 Inside the code **18**

A The usage summary **19**

1 What EnCorelib computes

EnCorelib takes an autonomous system of ordinary differential equations

$$x' = f(x), \qquad x \in \mathbb{R}^n,$$

a box $B_0 \subset \mathbb{R}^n$ of initial values, a horizon $H > 0$ and an error bound $\varepsilon > 0$. It returns a finite set C of boxes with

$$\mathrm{End}(B_0, H) \;\subseteq\; \bigcup C \;\subseteq\; \mathrm{End}(B_0, H) + [-\varepsilon, \varepsilon]^n, \tag{1}$$

where $\mathrm{End}(B_0, H) = \{\, x(H) : x(0) \in B_0 \,\}$ is the set of states reached at time H. A set C with this property is an *ε-cover* of the end set. The left inclusion says that no solution starting in B_0 is lost. The right one holds because every box of C is at most ε wide and contains a point of the end set. All arithmetic is interval arithmetic with directed rounding, so the inclusions hold for the boxes the program prints, rounding errors included. In one case the right inclusion can fail: when Refine cannot reach ε for a sub-box even after shrinking it to a single point, the wider box is kept, so that nothing is lost, and the program prints a note saying so.

With most validated solvers you learn the width of the result after the run. Here ε is an input: EnCorelib subdivides B_0 until every piece has an end-enclosure at most ε wide, and C is the set of these end-enclosures. The program also reports $B_1 = \mathrm{Box}(C)$, the smallest box that contains the cover. Because every box of C lies within ε of the end set, B_1 exceeds the smallest box around $\mathrm{End}(B_0, H)$ by at most ε on each side.

The cover can be much smaller than B_1. Figure 1 shows Eijgenraam's example $x' = 0$, $y' = x^2$ with $x(0) \in [-1, 1]$ and $y(0) = 0$. At $H = 1$ the end set is the arc $y = x^2$, $|x| \le 1$. Every convex set that contains the arc also contains $(0, 1)$, the midpoint of its two end points, although no solution comes near that point. The cover for $\varepsilon = 0.1$ has 79 boxes and follows the arc.

The algorithm is the one of B. Zhang and C. Yap, *End Cover for Initial Value Problem: Complete Validated Algorithm with Complexity Analysis* (arXiv:2502.00503 and arXiv:2602.00162), called *the paper* below. Section and equation numbers such as (5.5) refer to it. Section 9 shows where each part of the paper sits in the code. Everything runs in IEEE double precision.

2 Installing

2.1 What you need

- A C++17 compiler: g++ 9 or newer, or clang++ 10 or newer.

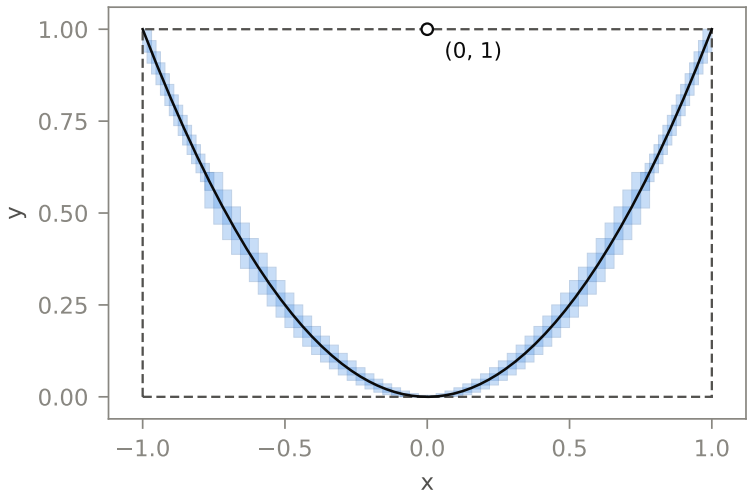


Figure 1: Eijgenraam's example at $H = 1$: the 0.1-cover (79 boxes), the end set $y = x^2$ (black) and the bounding box B_1 of the cover (dashed). The point $(0, 1)$ lies in B_1 and in every convex enclosure of the arc. Data from `make run EG=examples/fig1Eijgenraam.mk`.

- GNU make and bash, for the Makefile and the scripts in `repro/` and `doc/`. CMake 3.16 or newer if you prefer the CMake build.
- CAPD (`http://capd.ii.uj.edu.pl`), which supplies the interval arithmetic and the Taylor coefficients of f.
- Optional: SymEngine, which multiplies out the right-hand side before it is evaluated (Section 4.4); Python 3 with matplotlib for the plots; MATLAB for `tools/pendulum_ode45.m` only.

On Windows, install Cygwin with the packages `gcc-g++`, `make`, `cmake` and `git`, and type every command of this guide in a Cygwin login shell. Build EnCorelib in the same shell that built CAPD. Inside Cygwin the Windows path `C:\Users\you\CAPD` is written `/cygdrive/c/Users/you/CAPD`.

2.2 Building CAPD

Clone CAPD wherever you like and build it; you will need the path in the next step.

```
cd ~
git clone https://github.com/CAPDGroup/CAPD.git
cd CAPD && mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release -DCAPD_BUILD_EXAMPLES=OFF
↪ -DCAPD_BUILD_TESTS=OFF
make -j4
```

This leaves `libcapd.a` in `~/CAPD/build`. Running `make install` is optional.

2.3 Building EnCorelib

EnCorelib reads the location of CAPD from a file `config.mk` in its top directory. Copy the template and change its one active line to the absolute path of the directory you cloned CAPD into (the one that contains `capdDynSys/` and `build/`):

```
cp config.mk.example config.mk
# in config.mk:
CAPD_MASTER := /home/you/CAPD
```

`config.mk` is ignored by git, so the path stays on your machine. You can also give the path on each call, `make CAPD_MASTER=/home/you/CAPD`, or point at an installed CAPD with `make CAPD_CONFIG=/usr/local/bin/capd-config`. If `capd-config` is on your `PATH`, a plain `make` finds it.

Now run `make`. The first line it prints names the CAPD it uses and the last one is `Built EnCorelib`. If it says `CAPD not found`, check that `build/libcapd.a` exists under the directory you gave. Three companion programs are built on request: `make Encoretraj`, a validated trajectory printer used by `make verify`; `make capdref`, the plain CAPD integrator behind the CAPD rows of the paper's Table 2; and `make Encoreaudit`, the box-by-box check of Section 8. `make tools` builds all three.

There is also a CMake build:

```
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
cmake --build . --parallel
ctest
```

It uses `capd-config` when it finds one and otherwise downloads CAPD from GitHub and builds it first. SymEngine is used only with `-DUSE_SYMENGINE=ON`. Both builds compile with `-frounding-math`. Keep it: without it the compiler may assume round-to-nearest and break the directed rounding that every interval bound relies on.

CAPD can be built with the interval type of the filib library, which its CMake build does by default, or with its own interval type. EnCorelib uses only operations that both provide and compiles against either.

2.4 Checking the build

`make check` runs 18 short computations and compares B_1, rounded to two decimals, and the number of boxes $\#(C)$ with stored values. It takes about twenty seconds and should end like this:

```
Boundary mode
ok    Eg1 boundary 2D  H=1                  (0.08, 1.46) +- (0.01, 0.07)  #(C)=4
ok    Eg6 boundary 3D  H=1                  (-1.74, 1.86, 0.03) +- (0.15, 0.18, 0.00)
↪  #(C)=6

18 passed, 0 failed
```

The stored values hold whether or not SymEngine was found (Section 4.4 explains why that could matter).

3 A first run

3.1 An example from the paper

The directory `examples/` holds the test problems of the paper's Table 1 as `.mk` files. Eg1 is the Lotka-Volterra system $x' = 2x(1-y)$, $y' = -y(1-x)$ from $B_0 = [0.9, 1.1] \times [2.9, 3.1]$, with $H = 4$ and $\varepsilon = 1$ in the file:

```
$ make run-eg1Volterra
EnCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (2*x-2*x*y, -y+x*y)
         B0 = {[0.9, 1.1],[2.9, 3.1]}   H=4   eps=1
time(ms)=1029
```

```
Hull(T=4.000000)={[1.093880, 1.853071],[0.146098, 0.243519]}
B1=(1.473475, 0.194809) +- (0.379596, 0.048710)
wmax(B1)=0.759191
#(C)=5  #E0=5  max_{B in C} wmax(B)=0.551660
```

In the examples of this guide a line that starts with `$` is what was typed; the lines below it are what the program printed. Reading the output from the top:

`EnCorelib mode=…`
: the settings in force. The file says `tubedegree=0`, which selects the Euler tube, printed as `tubedegree=1`.

`f =`
: the right-hand side in the form in which it is evaluated (Section 4.4).

`B0, H, eps`
: the initial box, the horizon and ε.

`time(ms)`
: the wall-clock time of the computation in milliseconds.

`Hull(T=4.000000)`
: the box $B_1 = \mathrm{Box}(C)$, as one interval per coordinate.

`B1` the same box as midpoint ± half-width, the form of the B_1 column of the paper's Table 2.

`wmax(B1)`
: the largest side of B_1.

`#(C)`
: the number of boxes in the cover. `#E0` counts the sub-boxes of B_0 that were integrated, one for each box of C, and the last number is the largest side among the boxes of C, which is at most ε.

Here C has 5 boxes, none wider than 0.552, and B_1 is 0.759 wide in x and 0.097 wide in y.

3.2 The same run without make

`make` passes the fields of the `.mk` file to the program as positional arguments. `make show-args` prints the command instead of running it:

```
$ make show-args EG=examples/eg1Volterra.mk
./EnCorelib 2 0 0 0 0 2 'x' 'y' '2*x-2*x*y' '-y+x*y' 1.0 20 4.0 0 0.90000000000000002
↪  1.1000000000000001 2.8999999999999999 3.1000000000000001
```

The quotes are only there for the shell. Typed by hand, the same run is

```
$ ./EnCorelib 2 0 0 0 0 2 x y 2*x-2*x*y -y+x*y 1.0 20 4 0  0.9 1.1  2.9 3.1
```

and it prints the same lines as `make run-eg1Volterra`. Table 1 lists the arguments in order. Section 6 discusses how to choose them, and the program prints a short summary when it is called without arguments (Appendix A).

3.3 Output levels and files

`output_mode` is cumulative: each level prints what the levels below it print and adds the items of Table 2. Level 2 is the usual choice. The files are written into the current directory, and every run appends one line to `out.txt`.

Here is Eg1 again with $\varepsilon = 0.1$ at level 3, followed by the first lines of `E1.txt` and the last two lines of the log:

Table 1: The command-line arguments of `EnCorelib`, in order.

argument	meaning
`output_mode`	how much is printed and written, 0 to 4 (Table 2)
`mode`	0: EndCover, cover all of B_0; 1: Boundary, cover only the boundary of B_0
`method`	the refinement operators Refine may use: 0 `tube`, 1 `bisect`, 2 `splitonly`
`stepB`	the one-step enclosure: 0 `lohner`, 1 `lohner+lognorm`, 2 `lohnerHO+lognorm`, 3 `lohnerHO`
`tubedegree`	0 or 1, both the Euler tube of the paper's Section 5.2; no other value is accepted
`n`	the dimension n
`var_1 … var_n`	the names of the variables
`f_1 … f_n`	the right-hand sides (Section 4.2)
`eps`	the error bound $\varepsilon > 0$
`order`	the Taylor order of StepB, at least 3; the paper uses 20
`T`	the horizon $H > 0$
`debug`	progress reports, 0 to 3 (Section 5)
`lo_1 hi_1 … lo_n hi_n`	the initial box B_0

Table 2: What each output level adds, and the files it writes.

level	adds
0	`time(ms)`
1	the header lines, `Hull`, `B1` and `wmax(B1)`
2	`#(C)`, `#E0` and the largest side among the boxes of C
3	`E0.txt`: the sub-boxes of B_0 that were integrated, in the order of C; `E1.txt`: the cover C; in two dimensions `convex_hull.txt`, the convex hull of C as a closed polygon
4	`E_0.txt`, `E_1.txt`: the same, plus the images of every sub-box at $t = 0.1$, 0.4 and 0.7 and, in two dimensions, the trajectories of the four corners of B_0, for plotting

```
$ ./EnCorelib 3 0 0 0 0 2 x y 2*x-2*x*y -y+x*y 0.1 20 4 0  0.9 1.1  2.9 3.1
EnCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (2*x-2*x*y, -y+x*y)
         B0 = {[0.9, 1.1],[2.9, 3.1]}   H=4   eps=0.1
time(ms)=30646
Hull(T=4.000000)={[1.201730, 1.718819],[0.165442, 0.216428]}
B1=(1.460274, 0.190935) +- (0.258544, 0.025493)
wmax(B1)=0.517089
#(C)=85  #E0=85  max_{B in C} wmax(B)=0.079347
written -> E0.txt (85 boxes)
written -> E1.txt (85 boxes)
convex hull -> convex_hull.txt (20 pts)
```

```
$ head -n 3 E1.txt
Box 0: [1.4121234154389585e+00, 1.4810807664167143e+00] x [1.8370496338603454e-01,
↪  1.9073296662371442e-01]
Box 1: [1.2899831453924593e+00, 1.3551620435469465e+00] x [1.7281461672527448e-01,
↪  1.7888858491028295e-01]
Box 2: [1.2314825508867433e+00, 1.2949016537308344e+00] x [1.6792140651551760e-01,
↪  1.7359716618719925e-01]
```

```
$ tail -n 2 out.txt
EndCover  EndCover  tube  0  0.5  20  4  2  (1.41550692118,0.182810090845)  (0.32221794809
↪  7,0.0373248254251)  0.644435896195  2.474664  13  {[0.9, 1.1],[2.9,
↪  3.1]}
```

```
EndCover  EndCover  tube  0  0.1  20  4  2  (1.46027418606,0.19093464543)  (0.258544480614
↪  ,0.0254931111407)  0.517088961227  30.646121  85  {[0.9, 1.1],[2.9,
↪  3.1]}
```

A box file has one box per line, `Box k: [lo, hi] x [lo, hi] …`, with 17 significant digits; line k of `E0.txt` and line k of `E1.txt` belong together. The columns of `out.txt` are separated by tabs and named in its first line. Figure 2 draws the two box files. To draw them yourself, run

```
python3 tools/plot_boxes.py . --save cover.png
```

in the directory that holds them.

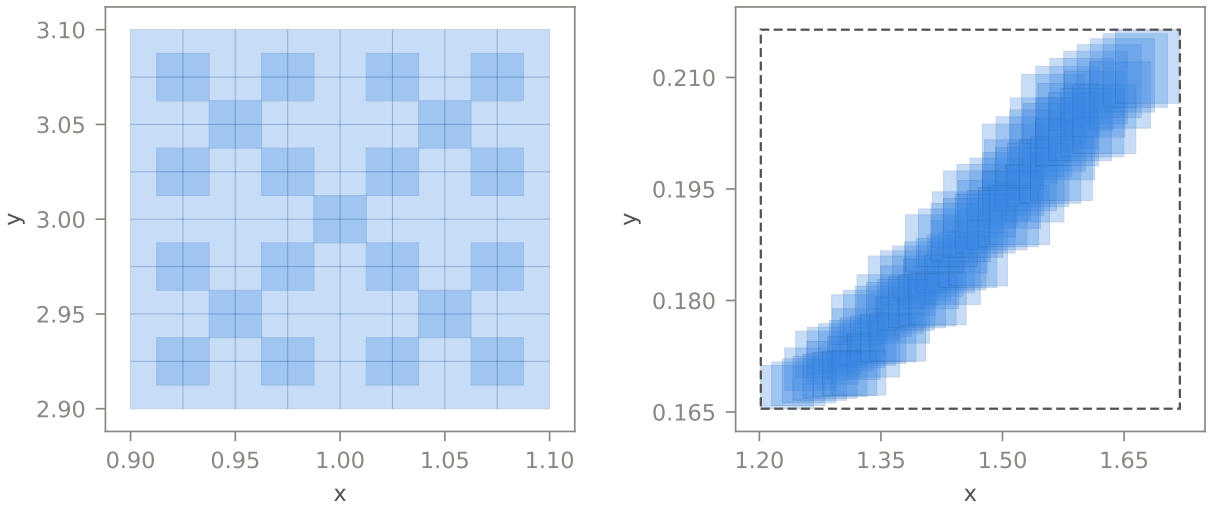


Figure 2: Eg1 with $\varepsilon = 0.1$ and $H = 4$. Left: the 85 sub-boxes of B_0 in `E0.txt`. Right: the 85 boxes of the cover C in `E1.txt` and their bounding box B_1 (dashed).

All 85 boxes of `E0.txt` are 0.025 wide. Sixty-four of them tile B_0; they are the sub-boxes of depth 3, which Refine resolved. The other 21 sit in the middle of the sub-boxes of depths 0, 1 and 2: they are the boxes to which Refine shrank E_0 before EndCover split those sub-boxes, and they show up darker where they overlap the tiling. Their end-enclosures belong to C as well.

4 A problem of your own

4.1 A `.mk` file

Take the damped pendulum $x' = y$, $y' = -\sin x - 0.2\,y$ from the box $[0.9, 1.1] \times [-0.1, 0.1]$ up to $H = 10$. The file `examples/dampedPendulum.mk` describes it:

```
# ============================================================================
#  A problem of your own -- the damped pendulum       (not in the paper)
#
#      x' = y
#      y' = -sin(x) - 0.2 y
#
#      B0 = Box(1,0)(0.1)         H = 10
#
#  The user guide (doc/userguide.pdf) builds this file step by step.
#      make run EG=examples/dampedPendulum.mk
#      make run EG=examples/dampedPendulum.mk eps=0.05 debug=1
```

```
# ========================================================================
n   = 2
var = x,y
ff  = y,-sin(x)-0.2*y

cen = 1.0,0.0
wid = 0.1,0.1

eps   = 0.1
order = 20
T     = 10

output_mode = 2
mode        = 0
method      = 0
stepB       = 0
tubedegree  = 0
debug       = 0
```

`var` and `ff` are comma-separated lists. `cen` and `wid` give the box as centre and half-width in each coordinate; `lohi = "0.9 1.1 -0.1 0.1"` would give it by its bounds instead. The other fields are the arguments of Table 1, and `examples/_template.txt` describes them all. Run the file with `make run`:

```
$ make run EG=examples/dampedPendulum.mk
EnCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (y, -0.2*y-sin(x))
         B0 = {[0.9, 1.1],[-0.1, 0.1]}   H=10   eps=0.1
time(ms)=2144
Hull(T=10.000000)={[-0.427619, -0.300730],[0.016225, 0.146630]}
B1=(-0.364174, 0.081428) +- (0.063444, 0.065202)
wmax(B1)=0.130405
#(C)=5  #E0=5  max_{B in C} wmax(B)=0.095115
```

Any field can be set on the command line without editing the file. Here ε is halved and the progress reports of Section 5 are switched on:

```
$ make run EG=examples/dampedPendulum.mk eps=0.05 debug=1
EnCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (y, -0.2*y-sin(x))
         B0 = {[0.9, 1.1],[-0.1, 0.1]}   H=10   eps=0.05
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     2.4s      1     0  2.0e-01   3.1e-02     4    0.0%  split, E_0 -> 5.0e-02
     4.1s      2     1  1.0e-01   3.2e-02     7    0.0%  split, E_0 -> 5.0e-02
     4.2s      3     2  5.0e-02   3.8e-02     6    6.2%  resolved
     4.3s      4     2  5.0e-02   3.3e-02     5   12.5%  resolved
     4.4s      5     2  5.0e-02   3.6e-02     4   18.7%  resolved
     4.5s      6     2  5.0e-02   3.2e-02     3   25.0%  resolved
     6.1s      7     1  1.0e-01   2.8e-02     6   25.0%  split, E_0 -> 5.0e-02
     6.2s      8     2  5.0e-02   3.2e-02     5   31.2%  resolved
     6.3s      9     2  5.0e-02   3.2e-02     4   37.5%  resolved
     6.4s     10     2  5.0e-02   3.2e-02     3   43.8%  resolved
     6.5s     11     2  5.0e-02   3.3e-02     2   50.0%  resolved
     8.2s     12     1  1.0e-01   3.1e-02     5   50.0%  split, E_0 -> 5.0e-02
     8.3s     13     2  5.0e-02   3.7e-02     4   56.2%  resolved
     8.4s     14     2  5.0e-02   3.2e-02     3   62.5%  resolved
     8.5s     15     2  5.0e-02   3.7e-02     2   68.8%  resolved
     8.7s     16     2  5.0e-02   3.3e-02     1   75.0%  resolved
    10.2s     17     1  1.0e-01   2.9e-02     4   75.0%  split, E_0 -> 5.0e-02
    10.3s     18     2  5.0e-02   3.3e-02     3   81.3%  resolved
    10.4s     19     2  5.0e-02   3.3e-02     2   87.5%  resolved
    10.5s     20     2  5.0e-02   3.3e-02     1   93.8%  resolved
```

```
    10.6s     21     2  5.0e-02   3.4e-02     0  100.0%  resolved
    10.6s  EndCover finished: 21 sub-boxes processed, 16 resolved, deepest level 2, #(C)=21
time(ms)=10598
Hull(T=10.000000)={[-0.409744, -0.304667],[0.024681, 0.133902]}
B1=(-0.357206, 0.079292) +- (0.052538, 0.054610)
wmax(B1)=0.109220
#(C)=21  #E0=21  max_{B in C} wmax(B)=0.038022
```

4.2 Writing the right-hand side

An f_i may use `+ - * /`, parentheses, powers `x^3`, and the functions `sin`, `cos`, `exp`, `log`, `sqrt`, `sqr`, `atan`, `asin` and `acos`. A number directly followed by a name or an opening parenthesis is multiplied by it, so `2x` is `2*x` and `(x+1)(y-1)` is `(x+1)*(y-1)`; digits inside a name, as in `x1` or `k2a`, belong to the name. In a `.mk` file the commas separate the f_i, so an f_i cannot contain one. On the command line each f_i is a separate argument: quote it when it contains parentheses or other characters the shell would interpret,

```
$ ./EnCorelib 1 0 0 0 0 2 x y y "-sin(x)-2e-1*y" 0.1 20 10 0  0.9 1.1  -0.1 0.1
```

4.3 Constants

A decimal constant such as `0.2` is read as the double nearest to it and enters the computation as an interval of width zero. The cover is then rigorous for the system whose coefficient is that double, which differs from 0.2 by about 10^{-17}. If the coefficient is meant to be exactly $1/5$, write `1/5`: a quotient is evaluated in interval arithmetic and its enclosure contains the exact value, and the same holds for `8/3` in the Lorenz example. Constants in scientific notation are turned into such quotients, `2e-1` into `(2/10^1)`. The header of the run above shows the result:

```
EnCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (y, (-1/5)*y-sin(x))
         B0 = {[0.9, 1.1],[-0.1, 0.1]}   H=10   eps=0.1
time(ms)=1930
Hull(T=10.000000)={[-0.427619, -0.300730],[0.016225, 0.146630]}
B1=(-0.364174, 0.081428) +- (0.063444, 0.065202)
wmax(B1)=0.130405
```

4.4 The form of the expression

Interval evaluation depends on how an expression is written: $x(1-x)$ and $x-x^2$ are the same function but give different enclosures over a box. If SymEngine was found when EnCorelib was built, every f_i is first multiplied out into a sum of monomials, so that `(1-x^2)*y-x` becomes `-x+y-x^2*y`. Without SymEngine it is evaluated as written. The `f =` line of the output shows the form that was used.

On Lorenz (Eg5) the difference is large. Run to $H = 5$ with $\varepsilon = 1$ and `stepB=0`, written as `10*(y-x), x*(28-z)-y, x*y-8*z/3` and evaluated as written, EndCover subdivided B_0 down to depth 3 and processed 585 sub-boxes. Multiplied out, it stopped at depth 2 after 73 sub-boxes, about eight times less work. The files in `examples/` and the scripts in `repro/` therefore spell out the multiplied-out form. That is also the form the numbers of the paper were computed from, so these files give the same results in builds with and without SymEngine.

4.5 Mistakes in the input

A wrong argument count, a negative ε or a box with `lo > hi` stops the program with `Argument error` and the usage summary. A right-hand side that cannot be read is reported with the map string the parser was given; here `Y` should have been `y`:

```
$ ./EnCorelib 1 0 0 0 0 2 x y y "-sin(x)-0.2*Y" 0.1 20 10 0  0.9 1.1  -0.1 0.1
Cannot read the right-hand side var:x,y;fun:y,-0.2*Y-sin(x);
undefined symbol 'Y' in the expression!
If this is a numerical constant in scientific notation please replace it by a parameter.
```

Both cases end with exit status 2.

5 Following a long run

A run can take hours, and with `debug=0` it prints nothing until it is finished. The `debug` argument switches on progress reports:

`debug`	reports
0	none
1	one line for every sub-box of B_0 that EndCover has processed
2	also one line for every scaffold laid out to the horizon and for every pass of Refine
3	also one line for every stage appended by Extend, and the containment self-checks, which should never fire

Each line starts with the wall-clock time since the program started. The reports go to standard error and the results to standard output, so `> result.txt` still captures the results alone, and `2> progress.log` keeps the reports in a file you can follow with `tail -f progress.log`.

5.1 One line per sub-box

Eg1 with $\varepsilon = 0.5$ at `debug=1`:

```
$ ./EnCorelib 2 0 0 0 0 2 x y 2*x-2*x*y -y+x*y 0.5 20 4 1  0.9 1.1  2.9 3.1
EnCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (2*x-2*x*y, -y+x*y)
         B0 = {[0.9, 1.1],[2.9, 3.1]}   H=4   eps=0.5
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     0.6s      1     0  2.0e-01   4.1e-01     4    0.0%  split, E_0 -> 1.0e-01
     0.7s      2     1  1.0e-01   4.6e-01     3   25.0%  resolved
     0.9s      3     1  1.0e-01   4.8e-01     2   50.0%  resolved
     1.2s      4     1  1.0e-01   1.6e-01     5   50.0%  split, E_0 -> 5.0e-02
     1.3s      5     2  5.0e-02   1.7e-01     4   56.2%  resolved
     1.4s      6     2  5.0e-02   1.7e-01     3   62.5%  resolved
     1.5s      7     2  5.0e-02   1.8e-01     2   68.7%  resolved
     1.6s      8     2  5.0e-02   1.8e-01     1   75.0%  resolved
     2.1s      9     1  1.0e-01   1.7e-01     4   75.0%  split, E_0 -> 5.0e-02
     2.2s     10     2  5.0e-02   1.7e-01     3   81.2%  resolved
     2.3s     11     2  5.0e-02   1.8e-01     2   87.5%  resolved
     2.4s     12     2  5.0e-02   1.8e-01     1   93.8%  resolved
     2.5s     13     2  5.0e-02   1.9e-01     0  100.0%  resolved
     2.5s  EndCover finished: 13 sub-boxes processed, 10 resolved, deepest level 2, #(C)=13
time(ms)=2504
Hull(T=4.000000)={[1.093289, 1.737725],[0.145485, 0.220135]}
B1=(1.415507, 0.182810) +- (0.322218, 0.037325)
wmax(B1)=0.644436
#(C)=13  #E0=13  max_{B in C} wmax(B)=0.481747
```

Each sub-box gets one line, written when its scaffold is finished:

`run`
: the number of the sub-box, in the order processed.

`depth`
: how often B_0 was subdivided to obtain it; B_0 itself has depth 0. A subdivision halves every coordinate of positive width and gives 2^n children, so $w_{\max}(B)$ halves with each level.

`wmax(E_m)`
: the width of its end-enclosure at H.

`queue`
: the sub-boxes still waiting.

`done`
: the share of B_0, by volume, covered by resolved sub-boxes.

`outcome`
: `resolved`: Refine reached ε for the whole sub-box, which is finished. `split, E_0 -> w`: Refine reached ε only after shrinking E_0 to width w; the part of the sub-box outside E_0 is not covered yet, so the sub-box is replaced by its 2^n children. `failed:` the scaffold could not be carried to the horizon, for the reason given, and the sub-box is subdivided as well (Section 7).

EndCover always takes the sub-box queued last, so it works depth first: the children of a split box are processed before its siblings. In this run Refine shrank E_0 in runs 1, 4 and 9, and each of those sub-boxes was replaced by its four children. The other ten sub-boxes were resolved, and the 13 end-enclosures form C.

5.2 Inside one sub-box

At `debug=2` each sub-box also shows how its scaffold was built and refined. The first lines of the same run:

```
$ ./EnCorelib 2 0 0 0 0 2 x y 2*x-2*x*y -y+x*y 0.5 20 4 2  0.9 1.1  2.9 3.1
EnCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (2*x-2*x*y, -y+x*y)
         B0 = {[0.9, 1.1],[2.9, 3.1]}   H=4   eps=0.5
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     0.2s      scaffold laid out to t=4: 55 stages, wmax(E_0)=2.0e-01 wmax(E_m)=6.3e+00
     0.5s      Refine pass 1: wmax(E_m)=4.1e-01 wmax(E_0)=1.0e-01 mu_max=2.9 m=55
     ↪  bisected=55 frozen=0 splits=1
     0.5s      1     0  2.0e-01   4.1e-01     4    0.0%  split, E_0 -> 1.0e-01
     0.7s      scaffold laid out to t=4: 44 stages, wmax(E_0)=1.0e-01 wmax(E_m)=4.6e-01
     0.7s      2     1  1.0e-01   4.6e-01     3   25.0%  resolved
     0.8s      scaffold laid out to t=4: 43 stages, wmax(E_0)=1.0e-01 wmax(E_m)=4.8e-01
     0.8s      3     1  1.0e-01   4.8e-01     2   50.0%  resolved
     0.9s      scaffold laid out to t=4: 44 stages, wmax(E_0)=1.0e-01 wmax(E_m)=5.2e-01
     1.2s      Refine pass 1: wmax(E_m)=1.6e-01 wmax(E_0)=5.0e-02 mu_max=2.74 m=44
     ↪  bisected=44 frozen=0 splits=1
     1.2s      4     1  1.0e-01   1.6e-01     5   50.0%  split, E_0 -> 5.0e-02
```

`scaffold laid out` means that Extend has reached $t = H$ in m stages; the widths of E_0 and of the end-enclosure E_m follow. Each `Refine pass` line reports one pass of the main loop of Refine: the new widths of E_m and E_0; `mu_max`, the logarithmic-norm bound that enters the tests (5.4) and (5.5); the number of stages m; how many stages `Bisect` refined in this pass; how many stages are frozen by (5.4); and how many times `Split` shrank E_0. For run 1, Extend reached $H = 4$ in 55 stages with an end-enclosure 6.3 wide. One pass of Refine bisected all 55 stages and split once, which halved E_0 to 0.1 and brought E_m down to 0.41, below $\varepsilon = 0.5$. In runs 2 and 3 the end-enclosure was within ε as soon as the scaffold reached H, so Refine had nothing to do.

At `debug=3` every stage appears as Extend appends it, with its end time t_i, its length h and its end-enclosure E_i:

```
$ ./EnCorelib 2 0 0 0 0 2 x y 2*x-2*x*y -y+x*y 0.5 20 4 3  0.9 1.1  2.9 3.1
EnCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=2
         f  = (2*x-2*x*y, -y+x*y)
         B0 = {[0.9, 1.1],[2.9, 3.1]}   H=4   eps=0.5
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     0.0s        stage 1: t=0.0683917 h=6.8e-02 wmax(E)=2.4e-01 E={[0.670312,
     ↪  0.852826],[2.85598, 3.09299]}
     0.1s        stage 2: t=0.109407 h=4.1e-02 wmax(E)=2.5e-01 E={[0.563562,
     ↪  0.733611],[2.81151, 3.06553]}
     0.1s        stage 3: t=0.158235 h=4.9e-02 wmax(E)=2.7e-01 E={[0.459635,
     ↪  0.616735],[2.74636, 3.01459]}
     0.1s        stage 4: t=0.203157 h=4.5e-02 wmax(E)=2.8e-01 E={[0.381751,
     ↪  0.529877],[2.67632, 2.95557]}
     0.1s        stage 5: t=0.246126 h=4.3e-02 wmax(E)=2.9e-01 E={[0.320527,
     ↪  0.461663],[2.60361, 2.89022]}
     0.1s        stage 6: t=0.296907 h=5.1e-02 wmax(E)=2.9e-01 E={[0.262497,
     ↪  0.395432],[2.51361, 2.80423]}
     0.1s        stage 7: t=0.343782 h=4.7e-02 wmax(E)=2.9e-01 E={[0.219436,
     ↪  0.34597],[2.42739, 2.72002]}
```

Short stages mark the parts of $[0, H]$ where StepA or the error control of StepB allowed only small steps.

5.3 A long run

Lorenz (Eg5) to $H = 5$ with $\varepsilon = 1$, from the `.mk` file, begins

```
$ make run EG=examples/eg5Lorenz.mk T=5 debug=1
EnCorelib  mode=EndCover  method=tube  stepB=lohner  tubedegree=1  order=20  n=3
         f  = (-10*x+10*y, 28*x-y-x*z, (-8/3)*z+x*y)
         B0 = {[14.999, 15.001],[14.999, 15.001],[35.999, 36.001]}   H=5   eps=1
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     3.4s      1     0  2.0e-03         -     8    0.0%  failed: Extend: StepB cannot
     ↪  advance at t=3.79381 (H=5), split into 8
     7.2s      2     1  1.0e-03         -    15    0.0%  failed: Extend: StepB cannot
     ↪  advance at t=4.64434 (H=5), split into 8
     9.9s      3     2  5.0e-04   8.5e-01    14    1.6%  resolved
    12.7s      4     2  5.0e-04   8.5e-01    13    3.1%  resolved
    15.5s      5     2  5.0e-04   8.4e-01    12    4.7%  resolved
```

and ends

```
   203.8s     71     2  5.0e-04   8.7e-01     2   96.9%  resolved
   206.5s     72     2  5.0e-04   8.7e-01     1   98.4%  resolved
   209.2s     73     2  5.0e-04   9.5e-01     0  100.0%  resolved
   209.2s  EndCover finished: 73 sub-boxes processed, 64 resolved, deepest level 2, #(C)=64
time(ms)=209185
Hull(T=5.000000)={[1.016192, 1.744829],[1.850605, 3.008477],[16.076084, 17.058110]}
B1=(1.380511, 2.429541, 16.567097) +- (0.364319, 0.578936, 0.491013)
wmax(B1)=1.157872
#(C)=64  #E0=64  max_{B in C} wmax(B)=0.949437
```

The scaffold of B_0 could not be carried past $t = 3.79$, and each of its eight children stopped near $t = 4.65$. Their children, $5 \cdot 10^{-4}$ wide, all reached $H = 5$ with end-enclosures narrower than ε, and their 64 end-enclosures form C; a sub-box that failed contributes nothing. The `done` column moves by $1/64 \approx 1.6\%$ with each resolved sub-box, so after a few minutes of a long run it tells you roughly how much longer to wait.

6 Choosing the options

The inputs that decide what you get are ε and H. The other options decide how the work is done, and on the problems of the paper most of them change the time much more than the result. Every run below starts from the settings of the example files (`mode=0`, `method=0`, `stepB=0`, `tubedegree=0`, `order=20`), changes one option, and runs the problem to the longer horizon of the paper's Table 2. The runs come from `doc/compare_options.sh`, four at a time on an Intel Core i7-13700HX, and `doc/make_tables.py` turned them into the tables, so both can be regenerated. Times are wall-clock seconds. Running four at a time made otherwise identical runs differ by up to a factor of about two, so read only the larger differences in the tables, and compare times within a table rather than with your own machine. The last table was measured one run at a time.

6.1 The error bound

Table 3: Time and number of boxes as ε shrinks (all other options at their defaults).

		$\varepsilon = 1$		$\varepsilon = 0.5$		$\varepsilon = 0.1$	
problem	H	s	$\#(C)$	s	$\#(C)$	s	$\#(C)$
Eg1 Lotka-Volterra	5.5	18	64	18	64	173	524
Eg2 Van der Pol	2	1.1	8	1.6	12	20	104
Eg3 quadratic	4	0.7	17	1.2	25	36	293
Eg4 FitzHugh-Nagumo	4	1.0	4	1.1	4	2.4	16
Eg5 Lorenz	4	14	8	14	8	504	72
Eg6 Rössler	4	0.2	1	0.2	1	283	585

Going from $\varepsilon = 1$ to $\varepsilon = 0.5$ changed little and often nothing (Table 3), because an end-enclosure already narrower than 0.5 satisfies both. From 0.5 to 0.1 the number of boxes grew by a factor of 4 to 12 on the first five problems and the time by a factor of 2 to 35. Rössler is the extreme case: without any subdivision its end-enclosure is 0.495 wide, so $\varepsilon = 0.5$ takes a fifth of a second, and $\varepsilon = 0.1$ takes 585 sub-boxes and almost five minutes. Every level of subdivision multiplies the number of sub-boxes by 2^n, so the cost depends on how far ε lies below the width of the undivided end-enclosure. Start with a generous ε, look at $\#(C)$ and the time, and lower it in steps.

6.2 The horizon

The horizon sets how far the end set has spread by the time the cover is taken. For Lotka-Volterra with $\varepsilon = 0.1$ the cover has 85 boxes at $H = 4$ and 524 at $H = 5.5$ (`make table2`). For a chaotic system the horizon also decides whether a run can finish at all, because every level of subdivision multiplies the work by 2^n and gains only a fixed amount of time; Section 7 shows how to read that amount off the progress report.

6.3 The enclosure step: `stepB`

Variants 1 and 2 intersect each enclosure with the ball of the paper's Lemma 2.1, and variants 2 and 3 use Lohner's method with a higher-order a priori enclosure. In every row of Table 4 variant 1 returned the cover of variant 0, and variant 3 that of variant 2, with the same number of boxes and the same B_1: the ball narrowed nothing. It did cost time in every row, a factor of 1.3 to 2.1 for variant 1 over variant 0 and of 2 to 5 for variant 2 over variant 3.

Table 4: The four StepB variants: time and $w_{\max}(B_1)$.

			time (s) for `stepB`				$w_{\max}(B_1)$ for `stepB`			
problem	H	ε	0	1	2	3	0	1	2	3
Eg1 Lotka-Volterra	5.5	0.5	18	29	23	8.2	1.074	1.074	0.967	0.967
Eg2 Van der Pol	2	0.5	1.6	2.7	2.8	0.9	0.715	0.715	0.685	0.685
Eg3 quadratic	4	0.5	1.2	2.3	2.5	0.9	0.861	0.861	0.858	0.858
Eg4 FitzHugh-Nagumo	4	0.5	1.1	1.4	0.2	0.1	0.170	0.170	0.207	0.207
Eg5 Lorenz	4	0.5	14	28	258	66	0.458	0.458	0.135	0.135
Eg6 Rössler	4	0.5	0.2	0.5	0.2	0.1	0.495	0.495	0.484	0.484
Eg1 Lotka-Volterra	5.5	0.1	173	311	247	74	0.849	0.849	0.841	0.841
Eg2 Van der Pol	2	0.1	20	37	30	9.2	0.610	0.610	0.609	0.609
Eg3 quadratic	4	0.1	36	73	39	13	0.803	0.803	0.803	0.803
Eg4 FitzHugh-Nagumo	4	0.1	2.4	4.2	1.0	0.5	0.158	0.158	0.167	0.167
Eg5 Lorenz	4	0.1	504	880	677	145	0.138	0.138	0.135	0.135
Eg6 Rössler	4	0.1	283	521	246	64	0.464	0.464	0.464	0.464
Eg5 Lorenz	5	1	116	221	128	39	1.158	1.158	0.953	0.953

That leaves variants 0 and 3. Variant 3 was the faster one in 12 of the 13 rows, mostly by a factor between 2 and 5. Its B_1 was narrower in 9 rows and the same to three digits in 2; on FitzHugh-Nagumo it was wider, by 6% at $\varepsilon = 0.1$ and 22% at $\varepsilon = 0.5$, with fewer boxes. The one row where it was slower is Lorenz at $\varepsilon = 0.5$: it subdivided down to 72 boxes where variant 0 stopped at 8 and took 66 seconds instead of 14, and it returned a hull 0.135 wide instead of 0.458. Variant 0 is the setting of the paper's experiments and of the example files; for a new problem, try variant 3 first.

6.4 The refinement operators: `method`

Table 5: The three refinement strategies: time and $w_{\max}(B_1)$.

			time (s) for `method`			$w_{\max}(B_1)$ for `method`		
problem	H	ε	0	1	2	0	1	2
Eg1 Lotka-Volterra	5.5	0.5	18	17	18	1.074	1.074	1.074
Eg2 Van der Pol	2	0.5	1.6	1.3	1.1	0.715	0.715	0.715
Eg3 quadratic	4	0.5	1.2	0.8	0.6	0.861	0.861	0.861
Eg4 FitzHugh-Nagumo	4	0.5	1.1	1.0	1.0	0.170	0.170	0.170
Eg5 Lorenz	4	0.5	14	14	14	0.458	0.458	0.458
Eg6 Rössler	4	0.5	0.2	0.2	0.2	0.495	0.495	0.495
Eg1 Lotka-Volterra	5.5	0.1	173	109	89	0.849	0.849	0.849
Eg2 Van der Pol	2	0.1	20	11	7.7	0.610	0.610	0.610
Eg3 quadratic	4	0.1	36	11	7.5	0.803	0.803	0.803
Eg4 FitzHugh-Nagumo	4	0.1	2.4	2.6	2.0	0.158	0.158	0.158
Eg5 Lorenz	4	0.1	504	145	102	0.138	0.138	0.138
Eg6 Rössler	4	0.1	283	243	157	0.464	0.464	0.464

The three strategies returned covers with the same number of boxes and the same B_1 in all twelve rows of Table 5. On these problems `Bisect` and `EulerTube` changed no result and only added time; `Split` did all the work. At $\varepsilon = 0.1$, method 2 needed between 20% and 83% of the time of method 0, and method 1 was usually in between. At $\varepsilon = 0.5$, where Refine has little to do, the three took about the same time. Method 0 is Refine as the paper states it, and the setting of its experiments; use method 2 when time matters.

6.5 The Taylor order

Table 6: The Taylor order of StepB: time and $w_{\max}(B_1)$.

			time (s) for `order`				$w_{\max}(B_1)$ for `order`			
problem	H	ε	10	15	20	30	10	15	20	30
Eg1 Lotka-Volterra	5.5	0.1	290	136	173	310	0.844	0.847	0.849	0.854
Eg5 Lorenz	4	0.5	59	183	14	255	0.404	0.132	0.458	0.162
Eg6 Rössler	4	0.1	440	303	283	260	0.464	0.464	0.464	0.464

The order changes the step sizes, and with them the whole run, in ways that are hard to predict (Table 6). On Lotka-Volterra the lowest order gave the narrowest hull with the fewest boxes but took twice as long as order 15. On Lorenz, orders 15 and 30 subdivided down to 72 boxes where orders 10 and 20 needed 9 and 8. On Rössler the cover hardly changed and order 30 was the fastest. Order 20, the paper's value and the one in the example files, was never the slowest. Try 15 or 30 when a run is too slow.

6.6 Boundary mode

Table 7: EndCover (`mode=0`) and Boundary mode (`mode=1`) at $\varepsilon = 0.1$.

		`mode=0` (EndCover)			`mode=1` (Boundary)		
problem	H	s	$\#(C)$	$w_{\max}(B_1)$	s	$\#(C)$	$w_{\max}(B_1)$
Eg1 Lotka-Volterra	5.5	173	524	0.849	42	37	0.852
Eg2 Van der Pol	2	20	104	0.610	39	20	0.609
Eg3 quadratic	4	36	293	0.803	17	26	0.808
Eg4 FitzHugh-Nagumo	4	2.4	16	0.158	1.9	7	0.161
Eg5 Lorenz	4	504	72	0.138	2060	126	0.124
Eg6 Rössler	4	283	585	0.464	213	254	0.465

Boundary mode covers only the boundary of B_0: its four edges in two dimensions, its $2n$ faces in n dimensions. The flow maps the boundary of B_0 onto the boundary of the end set, so the bounding box B_1 of this cover encloses $\mathrm{End}(B_0, H)$ as well; the boxes themselves do not cover the interior of the end set. In Table 7 boundary mode needed far fewer boxes, and its B_1 was within 2% of the one EndCover found, except on Lorenz, where it was 10% narrower. The time went both ways: four times faster on Lotka-Volterra, but twice as slow on Van der Pol and four times as slow on Lorenz. Use it when B_1 is all you need.

6.7 The remaining options

`tubedegree` has one meaningful value: 0 and 1 both select the Euler tube of the paper's Section 5.2, and the program refuses anything else. `output_mode=2` is right for most runs; level 3 writes the box files and level 4 also computes, for every sub-box of B_0, its images at $t = 0.1$, 0.4 and 0.7 (those not beyond H), which costs one more integration from each sub-box for each of these times. `debug=1` prints a line per sub-box and costs nothing you would notice, so it is worth switching on for anything that runs longer than a minute; levels 2 and 3 write far more and are meant for looking into one sub-box.

Table 8: The settings of the example files against `method=2` with `stepB=3`.

			the example files			`method=2 stepB=3`		
problem	H	ε	s	$\#(C)$	$w_{\max}(B_1)$	s	$\#(C)$	$w_{\max}(B_1)$
Eg1 Lotka-Volterra	5.5	0.1	106	524	0.849	21	383	0.841
Eg2 Van der Pol	2	0.1	12	104	0.610	2.1	93	0.609
Eg3 quadratic	4	0.1	24	293	0.803	3.7	293	0.803
Eg4 FitzHugh-Nagumo	4	0.1	1.8	16	0.158	0.3	4	0.167
Eg5 Lorenz	4	0.1	647	72	0.138	26	72	0.135
Eg6 Rössler	4	0.1	224	585	0.464	29	585	0.464
Eg5 Lorenz	5	1	85	64	1.158	27	64	0.953

6.8 In short

- Choose ε and H first. They decide the result and most of the cost. Lower ε in steps, and for a chaotic problem read the reachable horizon off the progress report (Section 7).
- Switch on `debug=1` for anything that runs longer than a minute.
- The example files use the settings of the paper's experiments, `method=0`, `stepB=0` and `order=20`; keep them when you compare with the paper's tables.
- For speed, use `method=2` and `stepB=3`. Run together and one at a time (Table 8), they were 3 to 25 times faster than the settings of the example files and returned the same or a narrower B_1, except on FitzHugh-Nagumo, where B_1 was 6% wider.
- Write the right-hand side multiplied out, or build with SymEngine (Section 4.4).
- Use `mode=1` only when B_1 is all you need.

7 When a run fails

A failure inside a run is not fatal. When the scaffold of a sub-box cannot be carried to H, typically because StepB cannot find a step for an enclosure that has grown too wide, EndCover subdivides the sub-box and tries its children; with `debug=1` such a sub-box shows up as `failed:` followed by the reason and the time reached. The Lorenz run of Section 5 had nine such failures before its 64 sub-boxes of depth 2 went through. A smaller box has a narrower enclosure, so the children usually get further, and the run itself fails only when there is nothing left to subdivide.

message	meaning and remedy
`Argument error:` … followed by the usage summary, exit status 2	an argument is missing or out of range; the message says which
`Cannot read the right-hand side …`, exit status 2	the parser rejected an f_i (Section 4.2)
`[EndCover failed] … The sub-box of B0 on which this happened is a single point … Try a smaller T.`, exit status 3	even the solution through one point could not be enclosed up to H, so subdividing cannot help. Lower T; see the example below
`[EndCover failed] EndCover: exceeded maxSplits …`, exit status 3	more than two million subdivisions; raise ε or lower T
`[Boundary failed] …`, exit status 3	the same for `mode=1`: an edge segment could not be enclosed even as a single point

The single-point failure is the one to expect with chaotic systems. Lorenz from the box of Eg5 to $H = 100$, with `stepB=3` and `debug=1`, starts

```
$ make run EG=examples/eg5Lorenz.mk T=100 stepB=3 debug=1
EnCorelib  mode=EndCover  method=tube  stepB=lohnerHO  tubedegree=1  order=20  n=3
         f  = (-10*x+10*y, 28*x-y-x*z, (-8/3)*z+x*y)
         B0 = {[14.999, 15.001],[14.999, 15.001],[35.999, 36.001]}   H=100   eps=1
     time    run depth  wmax(B) wmax(E_m) queue    done  outcome
     0.4s      1     0  2.0e-03         -     8    0.0%  failed: Extend: StepB cannot
     ↪  advance at t=3.83076 (H=100), split into 8
     0.8s      2     1  1.0e-03         -    15    0.0%  failed: Extend: StepB cannot
     ↪  advance at t=4.60846 (H=100), split into 8
     1.3s      3     2  5.0e-04         -    22    0.0%  failed: Extend: StepB cannot
     ↪  advance at t=5.45192 (H=100), split into 8
```

and ends

```
    52.6s     37    36  2.8e-14         -   260    0.0%  failed: Extend: StepB cannot
    ↪  advance at t=26.6501 (H=100), split into 8
    54.7s     38    37  1.4e-14         -   267    0.0%  failed: Extend: StepB cannot
    ↪  advance at t=25.9628 (H=100), split into 8
    57.1s     39    38  7.1e-15         -   270    0.0%  failed: Extend: StepB cannot
    ↪  advance at t=26.5727 (H=100), split into 4
    59.3s     40    39  7.1e-15         -   273    0.0%  failed: Extend: StepB cannot
    ↪  advance at t=26.5367 (H=100), split into 4
    61.5s     41    40  7.1e-15         -   272    0.0%  failed: Extend: StepB cannot
    ↪  advance at t=26.583 (H=100)
[EndCover failed] Extend: StepB cannot advance at t=26.583 (H=100)
The sub-box of B0 on which this happened is a single point (depth 40 of the subdivision),
so subdividing B0 further cannot help: the solution through that point
cannot be enclosed up to the horizon.
Try a smaller T.
make: *** [Makefile:194: run] Error 3
```

Every sub-box fails, so EndCover subdivides it and goes on with its last child, one level deeper each time. B_0 got to $t = 3.83$, and the sub-box at depth 20, a million times narrower, to $t = 19.13$: each halving bought about 0.76 time units, close to $\ln 2/0.906 \approx 0.765$, where 0.906 is the largest Lyapunov exponent of the Lorenz system. Beyond depth 30 the gain falls off, and from depth 33 on the failures stay between $t = 25.96$ and 26.65: at order 20 and in double precision the enclosure of a single solution cannot be carried further, whatever ε is. At depth 40 the sub-box is a single point and the program stops, after about a minute.

Long before that limit the cost becomes the problem, and the progress report lets you estimate it. Write t_d for the time that sub-boxes of depth d reach; in this run $t_0 = 3.83$, $t_1 = 4.61$, $t_2 = 5.45$, $t_3 = 6.21$ and $t_4 = 7.00$. Covering B_0 to the horizon H needs sub-boxes of a depth d with $t_d \geq H$, and there are 2nd of them. For Lorenz that means $8^2 = 64$ sub-boxes for $H = 5$ (the run of Section 5), presumably 512 for $H = 6$, and eight times more for every further 0.76 time units. A run that fails at about the same time while its depth keeps growing has met the precision limit; lower H.

8 Reproducing the paper

The tables and figures of the paper are regenerated by make targets in the top directory:

```
make table2      # the Ours(eps) rows of Table 2
make compare     # that run next to the printed table, cell by cell
make figure1     # the data of Figure 1
make figure4     # the data of Figure 4
make verify      # sample points of B0 checked against the covers
make audit       # every box of a cover checked against its own sub-box
```

`make table2` writes `repro/out/table2.txt` and `repro/out/table2.tex`. The variables `EPS`, `METHOD` and `STEPB` change its settings, and `bash repro/table2.sh eg1 eg5` restricts it to some of the problems. `make compare` reads `repro/table2_paper.txt`, which holds the table as printed, and marks every cell that differs. Timings depend on the machine; the paper's were taken on an Intel Core i7-13700HX with 16 GB of memory, and so were the timings in this guide.

`make figure1` computes Eijgenraam's example for $\varepsilon = 0.3$, 0.1 and 0.02 and the pendulum trajectories of the top row; `make figure4` writes `E0.txt` and `E1.txt` for each problem into `repro/out/fig4/`. The scripts `tools/plot_eijgenraam.py`, `tools/plot_pendulum.py` and `tools/plot_boxes.py` draw them; the README gives the exact commands.

`make verify` takes a grid of sample points of B_0 for each problem, encloses the solution through each point with `Encoretraj`, and checks that every such enclosure lies inside some box of C. This tests the left inclusion of (1), that nothing is lost. It does not test the right one, so a box of C that is far from the end set would pass. `make audit` checks the cover box by box: for every line k of `E0.txt` it encloses the solutions through the centre and the corners of that sub-box and checks that each enclosure meets line k of `E1.txt`. A box that fails cannot be an end-enclosure of its sub-box:

```
$ make audit
eps=0.1 stepB=0 order=20
eij  H=1   method=0  ok    79 boxes, 395 images: 0 boxes miss an image of their sub-box
eij  H=1   method=1  ok    79 boxes, 395 images: 0 boxes miss an image of their sub-box
eij  H=1   method=2  ok    79 boxes, 395 images: 0 boxes miss an image of their sub-box
eg1  H=4   method=0  ok    85 boxes, 425 images: 0 boxes miss an image of their sub-box
eg1  H=4   method=1  ok    85 boxes, 425 images: 0 boxes miss an image of their sub-box
eg1  H=4   method=2  ok    85 boxes, 425 images: 0 boxes miss an image of their sub-box
eg2  H=1   method=0  ok    65 boxes, 325 images: 0 boxes miss an image of their sub-box
eg2  H=1   method=1  ok    65 boxes, 325 images: 0 boxes miss an image of their sub-box
eg2  H=1   method=2  ok    65 boxes, 325 images: 0 boxes miss an image of their sub-box
eg3  H=1   method=0  ok    21 boxes, 105 images: 0 boxes miss an image of their sub-box
eg3  H=1   method=1  ok    21 boxes, 105 images: 0 boxes miss an image of their sub-box
eg3  H=1   method=2  ok    21 boxes, 105 images: 0 boxes miss an image of their sub-box
eg4  H=1   method=0  ok    20 boxes, 100 images: 0 boxes miss an image of their sub-box
eg4  H=1   method=1  ok    20 boxes, 100 images: 0 boxes miss an image of their sub-box
eg4  H=1   method=2  ok    20 boxes, 100 images: 0 boxes miss an image of their sub-box
eg5  H=1   method=0  ok    1 boxes, 9 images: 0 boxes miss an image of their sub-box
eg5  H=1   method=1  ok    1 boxes, 9 images: 0 boxes miss an image of their sub-box
eg5  H=1   method=2  ok    1 boxes, 9 images: 0 boxes miss an image of their sub-box
eg6  H=1   method=0  ok    73 boxes, 657 images: 0 boxes miss an image of their sub-box
eg6  H=1   method=1  ok    73 boxes, 657 images: 0 boxes miss an image of their sub-box
eg6  H=1   method=2  ok    73 boxes, 657 images: 0 boxes miss an image of their sub-box
```

9 Inside the code

The program is a set of headers and one main file. Table 9 says where each part of the paper is implemented; `CODE_SUMMARY.md` describes every function.

A run of `mode=0` goes through the files in this order. `main` reads the arguments and calls `EndCover` with B_0. EndCover keeps a queue of sub-boxes and calls `runScaffold` for each. `runScaffold` lays out a scaffold from the sub-box to $t = H$ with `ExtendToHorizon`, one stage per call of Extend, and then calls Refine. Each stage takes one step of StepB along a `SolutionChain`, whose length StepA bounds. Refine uses `Bisect`, `EulerTube` and `Split` as `method` allows and walks the chain again with `Evaluate` after each change. When Refine had to shrink E_0, EndCover queues the 2^n children of the sub-box; otherwise the sub-box is finished. Names in the code follow the paper: `S.E`, `S.F`, `S.G` and `S.t` are the components of a scaffold $S = (t, E, F, G)$, `eps0` is ε_0, and the tests (5.4), (5.5) and (5.6) are `StageFreeze`, `NeedSplit` and `CanTerminate`.

Table 9: The source files and the parts of the paper they implement.

file	contents	paper
`types.h`	boxes, $w_{\max}$, midpoints, the logarithmic norm, `Scaffold`, `MiniScaffold`	2.1, 2.4, 3.2, 5.1
`stepAB.h`	StepA, the four StepB variants, h_{euler}, and `SolutionChain`, which carries one Lohner doubleton along a scaffold	4.1, 4.2, (2.2), (2.6)
`extend.h`	Extend: append one stage; `ExtendToHorizon`, `RebuildScaffold`	4, 3.3
`refine.h`	Refine and its operators: `Evaluate`, `Bisect`, `EulerTube`, `Split`, and the tests (5.4) to (5.6)	5
`endcover.h`	EndCover: the queue of sub-boxes of B_0	3.3
`boundary.h`	`runScaffold`, which runs Extend and Refine for one sub-box, and the Boundary mode	3.3
`progress.h`	the progress reports of Section 5	
`symparse.h`	clean-up of the right-hand side (Section 4.2)	
`EnCorelib.cpp`	command line, output levels, output files	7

A The usage summary

```
$ ./EnCorelib
Argument error: too few arguments

Usage: EnCorelib output_mode mode method stepB tubedegree n \
               var1 ... varn  f1 ... fn \
               eps order T debug  lo1 hi1 ... lon hin

  output_mode 0=time 1=+hull 2=+counts 3=+E0/E1 files 4=+plot files
  mode        0=EndCover 1=Boundary
  method      0=tube (default) 1=bisect 2=splitonly
  stepB       0=lohner 1=lohner+lognorm 2=lohnerHO+lognorm 3=lohnerHO
  tubedegree  0 or 1: the Euler tube (the only one available)
  debug       0=silent 1=per sub-box 2=+Refine passes 3=+stages (on stderr)
```